UT: TRANSFORMACIÓN DE DOCUMENTOS. XSLT

Guión

- Introducción
- Estructura y sintaxis XSLT
- Diseño de plantillas
- Estructuras de control
- Creación de elementos

¿Qué es XSL?

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

XSL es una familia de lenguajes desarrollados por W3C que permiten describir cómo debe ser presentada la información contenida en un documento XML (stylesheet).

Las siguientes son las partes principales de XSL:

- **XSLT** : se utiliza para transformar documentos XML en otros tipos de documentos.
- **XPath** : se utiliza para navegar por el documento XML.
- **XSL-FO** : se utiliza para formatear documentos XML

¿Qué es XSLT?

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

- **XSLT (eXtensible Language for Stylesheet Transformations)**
- Es un estándar de la organización W3C que presenta una forma de transformar documentos XML en otros e incluso a formatos que no son XML (Wikipedia).
- XSLT es uno de los lenguajes derivados de XML, por tanto las hojas XSLT también son documentos XML (como también RSS, atom o los documentos XSD) y deben estar bien formados.

¿Qué transformaciones podemos realizar?

Introducción
Estructura y sintaxis XSLT
Diseño de plantillas
Estructuras de control
Creación de elementos

- **A otro documento XML.**
- **A un documento HTML.**
- A un documento de texto.
- Procesadores: Xalan o Saxon (Java), XMLStarlet o xsltproc (Linux), Clase XSLTProcessor (PHP)...

¿Como funciona?

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

- Utiliza **XPath** para navegar por las partes del documento.
- Contienen uno o varios elementos ***template***, que describen qué acción debe realizarse cuando se encuentra un determinado elemento o consulta XML en el archivo de destino
- El procesador XSLT toma la hoja de estilo XSLT y aplica las reglas de transformación (cuando se encuentra una coincidencia con el patrón, **transformará** la parte coincidente.) en el documento XML de destino y luego genera un documento formateado

¿Como funciona?

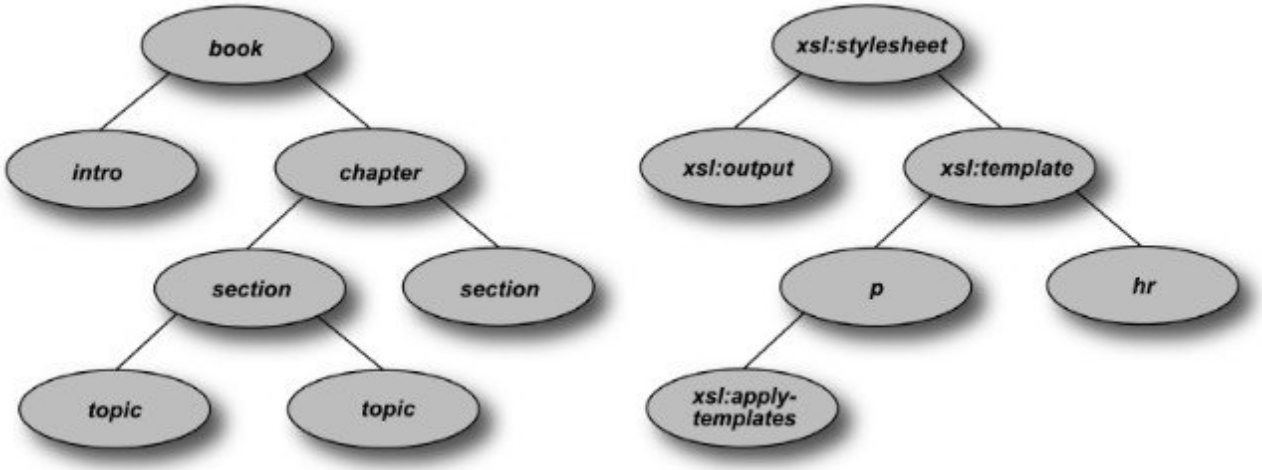
Introducción

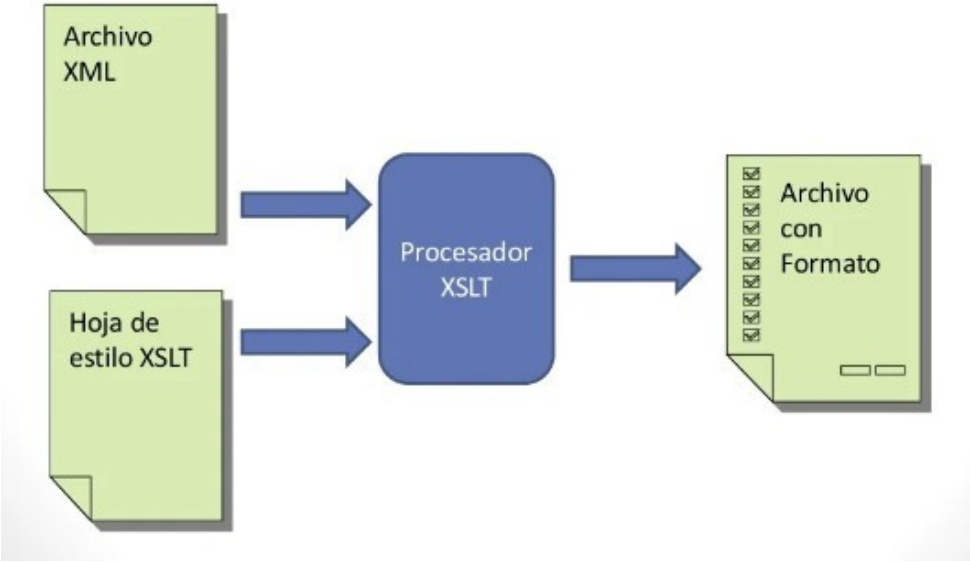
Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos





CSS vs XSLT

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

En común:

- Separación del contenido y de la presentación

CSS:

- El estilo y la presentación visual de conjunto

XSLT:

- El estilo y presentación visual (si se desea)
- Permite manipular el documento XML: reordenar, filtrar, añadir, borrar...
- Permite realizar cálculos y/o generar un valor
- Permite acceder a todo el documento, incluso atributos.

Elementos

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

- Una hoja de estilo XSLT es un documento XML y por lo tanto debe estar **bien formado**.
- Las hojas de estilo se guardarán siempre en archivos independientes con **extensión .xsl**
- Deben comenzar con una declaración XML:
  <?xml version="1.0" encoding="UTF-8"?>
- El elemento raíz de la hoja de estilo XSLT es stylsheet.

```
<xml:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

...

</xsl:stylesheet>
```

- Entre las marcas de inicio y de fin del elemento raíz, se escribirán las reglas de transformación que se llamarán plantillas.
- Cada regla se definirá mediante un elemento xsl:template (una plantilla).
- Se utiliza el atributo match para asociar una plantilla con un elemento XML

Ejemplo básico

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

Referencia al documento XSL

Archivo: starwars.xml

```
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <?xml-stylesheet type="text/xsl" href="starwars.xsl" ?>
3  <StarWars>
4   <pelicula>
5    <titulo>Star Wars: Episodio IV - Una nueva esperanza</titulo>
6    <fechaEstreno>25/05/1977</fechaEstreno>
7    <director>George Lucas</director>
8   </pelicula>
9   <pelicula>
10   <titulo>Star Wars: Episodio V - El Imperio contraataca</titulo>
11   <fechaEstreno>21/05/1980</fechaEstreno>
12   <director>irvin Kershener</director>
13  </pelicula>
14  <pelicula>
15   <titulo>Star Wars: Episode VI - El retorno del Jedi</titulo>
16   <fechaEstreno>25/05/1983</fechaEstreno>
17   <director>Richard Marqueand</director>
18  </pelicula>
19 </StarWars>
```

Archivo: starwars.xsl

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

<xsl:template match="//pelicula">
 <xsl:value-of select="director"/>
</xsl:template>

</xsl:stylesheet>
```

Cabecera

Obtener los directores de cada película

Etiqueta Templates

Introdución

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

<xsl:template>

La transformación se basa en el uso de plantillas (*templates*) con la etiqueta <xsl:template> que se aplicarán a cada nodo o atributo del documento según se va recorriendo éste, sustituyendo de esa manera el nodo por el contenido de su plantilla.

El atributo match indica los elementos afectados por la plantilla y contiene una expresión Xpath.

Plantillas no existentes

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos


```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
<xsl:output method="text"/>


</xsl:stylesheet>
```

Si no hay plantillas, el procesador recorre todos los nodos y extrae el contenido de cada nodo

Plantilla vacía

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos


```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
<xsl:output method="text"/>

    <xsl:template match="/Lugares/Lugar/Descripcion">
    </xsl:template>

</xsl:stylesheet>
```

Si hay una plantilla vacía, el procesador no genera ningún resultado en ese nodo en el documento final ni recorre los nodos hijos. Para el resto de nodos, mostrará el contenido.

¿Y si la plantilla vacía se aplica al elemento raíz?

Etiqueta value-of

Permite extraer el valor de un elemento XML y añadirlo a la salida

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transfor
m" version="1.0">
<xsl:output method="text"/>

    <xsl:template match="/Lugares/Lugar">
        <xsl:value-of select="Nombre"/>
        <xsl:value-of select="@codigo"/>
    </xsl:template>

</xsl:stylesheet>
```

Etiqueta apply-templates

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

Cuando se posiciona una plantilla a un nodo, en principio no se recorren los nodos descendientes. Para indicar que sí queremos recorrer los nodos descendientes y aplicarles las plantillas que les correspondan, hay que utilizar la instrucción <xsl:apply-templates />, como en el ejemplo siguiente

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">
<xsl:output method="text"/>

   <xsl:template match="/">
      <xsl:apply-templates select="//Lugar"/>
   </xsl:template>

<xsl:template match="//Lugar">
   <xsl:value-of select="Nombre" />
</xsl:template>

</xsl:stylesheet>
```

Etiqueta apply-templates

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

(file Lugares_IDNombre_tabla.xsl)

La instrucción <xsl:apply-templates> hace que se apliquen a los subelementos las reglas que les sean aplicables

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:output method="html"/>

<xsl:template match="/">
   <html>
   <head>
   <title>Ejemplo apply-templates</title>
   </head>
   <body>
   <table border="1">
   <tr><th>ID</th><th>NOMBRE</th></tr>
   <xsl:apply-templates select="//Lugar"/>
   </table>
   </body>
   </html>
</xsl:template>

<xsl:template match="//Lugar">
   <tr>
      <td>
         <xsl:value-of select="@codigo" />
      </td>
      <td>
         <xsl:value-of select="Nombre" />
      </td>
   </tr>
</xsl:template>
</xsl:stylesheet>
```

for-each

Permite pasar por todos los nodos seleccionados, como si se tratara de un bucle "for" en programación

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:output method="html"/>

<xsl:template match="/">
   <html>
   <head>
   <title>Ejemplo for-each</title>
   </head>
   <body>
   <table border="1">
   <tr><th>ID</th><th>NOMBRE</th></tr>
   <xsl:for-each select="Lugares/Lugar">
      <tr>
         <td>
            <xsl:value-of select="@codigo" />
         </td>
         <td>
            <xsl:value-of select="Nombre" />
         </td>
      </tr>

   </xsl:for-each>
   </table>
   </body>
   </html>
</xsl:template>
</xsl:stylesheet>
```

sort

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

Especifica un criterio de clasificación en los nodos.

```
<xsl:for-each select="Lugares/Lugar">
    <xsl:sort select = "@codigo"/>
   <tr>
      <td>
         <xsl:value-of select="@codigo" />
      </td>
      <td>
        <xsl:value-of select="Nombre" />
      </td>
   </tr>

</xsl:for-each>
```

if

Permite introducir una condición en la transformación mediante una condición evaluada mediante XPath en el atributo "test"

```
<xsl:for-each select="Lugares/Lugar">
  <xsl:if test="@codigo=5120">
   <tr>
      <td>
         <xsl:value-of select="@codigo" />
      </td>
      <td>
        <xsl:value-of select="Nombre" />
      </td>
   </tr>
   </xsl:if>
</xsl:for-each>
```

= (igual)
!= (no igual)
< (menor que)
> (mayor que)

Ejercicio: Adapta el documento par que muestre los lugares con la longitud esté entre 37,9 y 38

choose/when/otherwise

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

(file Lugares_choose.xsl)
Similar al **switch** de otros lenguajes, y permite condiciones consecutivas.

```
<xsl:choose>
  <xsl:when test="Subtipo='Museo'">
    ...
  </xsl:when>
  <xsl:when test="Subtipo='Plaza'">
     ...
  </xsl:when>
  <xsl:otherwise>
    ...
  </xsl:otherwise>
</xsl:choose>
```

Creación de elementos

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

Mediante *element* puede definirse un elemento nuevo.

```
<xsl:for-each select="Lugares/Lugar">
  <xsl:template match="poema ">
    <xsl:element name="h1">
      <xsl:value-of select="titulo"/>
  </xsl:element>
</xsl:for-each>
```

Puede especificarse el espacio de nombres del elemento mediante el atributo *namespace*.

Creación de elementos

Introducción

Estructura y sintaxis XSLT

Diseño de plantillas

Estructuras de control

Creación de elementos

Mediante *attribute* puede definirse un atributo nuevo.

```
<xsl:element name="img">
  <xsl:attribute select="src">
    <xsl:value-of select="@url"/>
  </xsl:attribute>
 <xsl:value-of select="."/>
</xsl:element>
```

<a href="mirul.com/imagen.jpg">Título de la imagen</a>

Mediante *text* se puede incluir texto

```
    <xsl:value-of select="nombre"/>
<xsl:text>, </xsl:text>
    <xsl:value-of select="apellido"/>
```

xsl:decimal-format

xsl:decimal-format

Define cómo se deben formatear los números cuando se usa la función format-number(). Permite controlar separadores decimales, separadores de grupos (miles), y otros aspectos del

```
<xsl:decimal-format
  name="nombre-formato"
  decimal-separator="carácter"
  grouping-separator="carácter"
  infinity="texto"
  minus-sign="carácter"
  NaN="texto"
  percent="carácter"/>
```

- name: Nombre del formato para referenciarlo
- decimal-separator: Carácter para separar decimales (ej. "," o ".")
- grouping-separator: Carácter para separar grupos de miles

xsl:number

xsl:number

Genera números en una secuencia o convierte valores a números formateados. Comúnmente usado para numerar elementos en una lista.

```
<xsl:number
  value="expresión"
  level="single|multiple|any"
  count="patrón"
  from="patrón"
  format="cadena-formato"/>
```

- value: Valor a numerar (opcional)
- level: Cómo contar (single, multiple o any)
- count: Qué elementos contar
- format: Formato del número (ej. "1", "a", "I")

Ejemplo - Archivo XML

Ejemplo: Catálogo de Raquetas - Archivo XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<raquetas>
  <raqueta>
    <marca>Wilson</marca>
    <modelo>Pro Staff</modelo>
    <precio>199.95</precio>
  </raqueta>
  <raqueta>
    <marca>Babolat</marca>
    <modelo>Pure Drive</modelo>
    <precio>229.5</precio>
  </raqueta>
  <raqueta>
    <marca>Head</marca>
    <modelo>Graphene 360</modelo>
    <precio>189.990</precio>
  </raqueta>
</raquetas>
```

Ejemplo XSLT - Formatos

Ejemplo XSLT - Definición de formatos numéricos

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <!-- Formato europeo: coma decimal, punto miles -->
  <xsl:decimal-format
    name="europeo"
    decimal-separator=","
    grouping-separator="."/>

  <!-- Formato americano: punto decimal, coma miles -->
  <xsl:decimal-format
    name="americano"
    decimal-separator="."
    grouping-separator=","/>

  <xsl:output method="html" encoding="UTF-8"/>
```

Ejemplo XSLT - Plantilla

Ejemplo XSLT - Uso en plantilla

```
<xsl:template match="raqueta">
  <tr>
    <!-- Numeración con xsl:number -->
    <td>
      <xsl:number level="single" count="raqueta"
                  format="1."/>
    </td>
    <td><xsl:value-of select="marca"/></td>
    <td><xsl:value-of select="modelo"/></td>

    <!-- Formato europeo -->
    <td>
      <xsl:value-of select="format-number(precio,
                    '#.##0,00', 'europeo')"/> €
    </td>

    <!-- Formato americano -->
    <td>
      <xsl:value-of select="format-number(precio,
                    '#,##0.00', 'americano')"/> $
    </td>
  </tr>
</xsl:template>
```

Salida HTML

Salida HTML generada

```
<tr>
  <td>1.</td>
  <td>Wilson</td>
  <td>Pro Staff</td>
  <td>199,95 €</td>
  <td>199.95 $</td>
</tr>
<tr>
  <td>2.</td>
  <td>Babolat</td>
  <td>Pure Drive</td>
  <td>229,50 €</td>
  <td>229.50 $</td>
</tr>
<tr>
  <td>3.</td>
  <td>Head</td>
  <td>Graphene 360</td>
  <td>189,99 €</td>
  <td>189.99 $</td>
</tr>
```

xsl:key

xsl:key

Define una clave para buscar y acceder eficientemente a nodos en un documento XML basándose en un valor específico. Útil para mejorar el rendimiento en transformaciones complejas.

```
<xsl:key
  name="nombre-clave"
  match="patrón"
  use="expresión"/>
```

Uso: key('nombre-clave', valor) para recuperar nodos

```
<!-- Ejemplo -->
generate-id(key('raquetas-por-marca', marca)[1])
```

Ejemplo xsl:key - XML

Ejemplo práctico xsl:key - Archivo XML

Librería con libros de diferentes editoriales:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<libreria>
  <libro id="L001">
    <titulo>El Quijote</titulo>
    <editorial>Planeta</editorial>
    <precio>25.99</precio>
  </libro>
  <libro id="L002">
    <titulo>1984</titulo>
    <editorial>Santillana</editorial>
    <precio>15.50</precio>
  </libro>
  <libro id="L003">
    <titulo>Cien años de soledad</titulo>
    <editorial>Planeta</editorial>
    <precio>22.00</precio>
  </libro>
  <libro id="L004">
    <titulo>La sombra del viento</titulo>
    <editorial>Planeta</editorial>
    <precio>19.95</precio>
  </libro>
</libreria>
```

Ejemplo xsl:key - XSLT

Ejemplo práctico xsl:key - Definición y uso

```
<!-- Definir la clave para buscar por editorial -->
<xsl:key name="libros-por-editorial"
         match="libro"
         use="editorial"/>

<xsl:template match="/">
  <html>
    <body>
      <h2>Libros agrupados por Editorial</h2>

      <!-- Seleccionar solo la primera aparición
           de cada editorial -->
      <xsl:for-each select="//libro[generate-id() =
                            generate-id(key('libros-por-editorial',
                            editorial)[1])]">

        <h3><xsl:value-of select="editorial"/></h3>
        <ul>
          <!-- Usar la clave para obtener todos los libros
               de esta editorial -->
          <xsl:for-each select="key('libros-por-editorial',
                                    editorial)">
            <li>
              <xsl:value-of select="titulo"/> -
              <xsl:value-of select="precio"/>€
            </li>
          </xsl:for-each>
        </ul>
      </xsl:for-each>
    </body>
  </html>
</xsl:template>
```

Ejemplo xsl:key - Resultado

Ejemplo xsl:key - Resultado

Los libros se agrupan automáticamente por editorial:

```
<h2>Libros agrupados por Editorial</h2>

<h3>Planeta</h3>
<ul>
  <li>El Quijote - 25.99€</li>
  <li>Cien años de soledad - 22.00€</li>
  <li>La sombra del viento - 19.95€</li>
</ul>

<h3>Santillana</h3>
<ul>
  <li>1984 - 15.50€</li>
</ul>
```

✓ La clave permite acceder eficientemente a todos los libros de una editorial sin recorrer todo el documento XML múltiples veces.

Patrón Muenchian explicado

Patrón Muenchian - Obtener valores únicos

La expresión que vimos es el "Patrón Muenchian", usado para obtener valores únicos:

```
//libro[generate-id() =
        generate-id(key('libros-por-editorial',
                        editorial)[1])]
```

Desglosando la expresión:

- //libro → Selecciona todos los libros del documento
- editorial → El valor de la editorial del libro actual
- key('libros-por-editorial', editorial) → Devuelve TODOS los libros con esa editorial
- [1] → Toma solo el PRIMERO de esos libros
- generate-id() → Genera un ID único para un nodo
- La comparación: ¿Este libro es el primero de su grupo?

Patrón Muenchian - Paso a paso

Patrón Muenchian - Ejemplo paso a paso

Tenemos 4 libros: L001(Planeta), L002(Santillana), L003(Planeta), L004(Planeta)

```
Evaluación para cada libro:

L001 (Planeta):
  key('libros-por-editorial', 'Planeta') → [L001, L003, L004]
  key(...)[1] → L001
  generate-id(L001) == generate-id(L001) → TRUE ✓
  → L001 es el PRIMERO de Planeta, se incluye

L002 (Santillana):
  key('libros-por-editorial', 'Santillana') → [L002]
  key(...)[1] → L002
  generate-id(L002) == generate-id(L002) → TRUE ✓
  → L002 es el PRIMERO de Santillana, se incluye

L003 (Planeta):
  key('libros-por-editorial', 'Planeta') → [L001, L003, L004]
  key(...)[1] → L001
  generate-id(L003) == generate-id(L001) → FALSE ✗
  → L003 NO es el primero, se DESCARTA

L004 (Planeta):
  key('libros-por-editorial', 'Planeta') → [L001, L003, L004]
  key(...)[1] → L001
  generate-id(L004) == generate-id(L001) → FALSE ✗
  → L004 NO es el primero, se DESCARTA
```

Patrón Muenchian - Resultado

Patrón Muenchian - Resultado

Al final solo se procesan los primeros de cada grupo:

```
Libros seleccionados:
  ✓ L001 (Planeta)   → Primero de "Planeta"
  ✓ L002 (Santillana) → Primero de "Santillana"
  ✗ L003 (Planeta)   → NO es el primero
  ✗ L004 (Planeta)   → NO es el primero
```

Ventajas del Patrón Muenchian:

- Obtiene valores únicos sin duplicados
- Muy eficiente: usa índices internos de xsl:key
- Permite agrupar elementos por cualquier criterio
- Estándar en XSLT 1.0 para agrupación

xsl:comment

xsl:comment

Agrega comentarios al documento de salida generado (HTML, XML, texto, etc.). Útil para documentar el código, depurar o proporcionar información adicional.

```
<xsl:comment>contenido del comentario</xsl:comment>
```

En HTML/XML se genera como: <!-- texto -->

El contenido puede incluir texto estático o valores dinámicos generados con <xsl:value-of>.

xsl:call-template

xsl:call-template

Invoca una plantilla con nombre (como una función). A diferencia de xsl:apply-templates, no depende del contexto XML sino que se llama explícitamente.

```
<!-- Definición -->
<xsl:template name="formatear">
  <xsl:param name="texto"/>
  <xsl:value-of select="concat('[', $texto, ']')"/>
</xsl:template>

<!-- Llamada -->
<xsl:call-template name="formatear">
  <xsl:with-param name="texto" select="."/>
</xsl:call-template>
```

xsl:param y xsl:with-param

xsl:param y xsl:with-param

xsl:param: Declara parámetros dentro de una plantilla con nombre.

```
<xsl:param name="prefijo" select="'['"/>
```

xsl:with-param: Pasa valores a los parámetros al llamar una plantilla.

```
<xsl:with-param name="prefijo" select="'**'"/>
```

Ejemplo call-template

Ejemplo completo con parámetros

```
<xsl:template name="formatear">
  <xsl:param name="texto"/>
  <xsl:param name="prefijo" select="'['"/>
  <xsl:param name="sufijo" select="']'"/>
  <xsl:value-of
    select="concat($prefijo, $texto, $sufijo)"/>
</xsl:template>

<xsl:template match="item">
  <xsl:call-template name="formatear">
    <xsl:with-param name="texto" select="."/>
    <xsl:with-param name="prefijo" select="'**'"/>
    <xsl:with-param name="sufijo" select="'**'"/>
  </xsl:call-template>
</xsl:template>
```